

Pandas

- Llibreria d'anàlisi de dades en Python.
- Ofereix DataFrames (estructures de dades tabulars).
- Potent per a la manipulació i exploració de dades.
- Normalment s'importa com a `pd`.

```
In [1]: import pandas as pd
```

DataFrames

- Els DataFrames es representen com a estructures bidimensionals.
- Tenen índexs, noms de columna i un valor per a cada parella índex-columna.
- Documentació: <https://pandas.pydata.org/docs/reference/frame.html>

```
In [2]: data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Frank", "Grace"],
    "Age": [22, 30, 35, 28, 35, 50, 22],
    "City": ["NY", "LA", "SF", "NY", "NY", "SF", "LA"],
    "Salary": [100, 200, 300, 250, 350, 500, 80],
    "Position": ["Engineer", "Manager", "CEO", "Engineer", "Manager", "Engineer", "Intern"],
    "Years of Experience": [2, 5, 10, 4, 12, 15, 1]
}

# Crea un DataFrame a partir de les dades d'un diccionari
df = pd.DataFrame(data)
df
```

```
Out[2]:
```

	Name	Age	City	Salary	Position	Years of Experience
0	Alice	22	NY	100	Engineer	2
1	Bob	30	LA	200	Manager	5
2	Charlie	35	SF	300	CEO	10
3	David	28	NY	250	Engineer	4
4	Eva	35	NY	350	Manager	12
5	Frank	50	SF	500	Engineer	15
6	Grace	22	LA	80	Intern	1

Bucles

```
In [3]: for idx, row in df.iterrows():  
        print(idx)  
        print('---')  
        print(row)  
        print()
```

```
0  
---  
Name                Alice  
Age                 22  
City               NY  
Salary             100  
Position           Engineer  
Years of Experience    2  
Name: 0, dtype: object
```

```
1  
---  
Name                Bob  
Age                 30  
City               LA  
Salary             200  
Position           Manager  
Years of Experience    5  
Name: 1, dtype: object
```

```
2
```

Aplicar funciones

- Realitza operacions sobre cada element.
- Més flexibilitat que les funcions integrades.

```
In [4]: # Aplicar una funció i crear una nova columna  
df['Age_squared'] = df['Age'].apply(lambda x: x**2)
```

```
In [5]: df
```

```
Out[5]:
```

	Name	Age	City	Salary	Position	Years of Experience	Age_squared
0	Alice	22	NY	100	Engineer	2	484
1	Bob	30	LA	200	Manager	5	900
2	Charlie	35	SF	300	CEO	10	1225
3	David	28	NY	250	Engineer	4	784
4	Eva	35	NY	350	Manager	12	1225
5	Frank	50	SF	500	Engineer	15	2500
6	Grace	22	LA	80	Intern	1	484

Agrupació de dades

- Agrupa per una o més columnes.
- Agrega utilitzant suma, mitjana, etc.

```
In [6]: # Agrupant per 'City' i obtenint l'edat mitjana  
grouped = df.groupby('City')['Age'].mean()  
print(grouped)
```

```
City  
LA      26.000000  
NY      28.333333  
SF      42.500000  
Name: Age, dtype: float64
```

```
In [7]: # Agrupant per 'City' i obtenint la suma del salari  
grouped = df.groupby('City')['Salary'].sum()  
print(grouped)
```

```
City  
LA      280  
NY      700  
SF      800  
Name: Salary, dtype: int64
```

```
In [8]: # Groupby múltiple amb size  
df.groupby(['City', 'Position']).size()
```

```
Out[8]:
```

City	Position	
LA	Intern	1
	Manager	1
NY	Engineer	2
	Manager	1
SF	CEO	1
	Engineer	1

dtype: int64

```
In [9]: # Groupby múltiple amb mediana  
df.groupby(['City', 'Position'])['Salary'].median()
```

```
Out[9]:
```

City	Position	
LA	Intern	80.0
	Manager	200.0
NY	Engineer	175.0
	Manager	350.0
SF	CEO	300.0
	Engineer	500.0

Name: Salary, dtype: float64

Dades absents

També conegudes com a `NA` o `NaN`.

In [16]:

```
df
```

Out[16]:

	key	value1	value2
0	A	1.0	4.0
1	B	2.0	5.0
2	C	3.0	NaN
3	D	NaN	6.0

Comprovar nuls

In [17]:

```
df.isna()
```

Out[17]:

	key	value1	value2
0	False	False	False
1	False	False	False
2	False	False	True
3	False	True	False

```
In [18]: # Columnes amb nuls  
df.isna().any(axis=0)
```

```
Out[18]: key      False  
value1    True  
value2    True  
dtype: bool
```

```
In [19]: # Comprovar columnes amb nuls  
df.loc[:, df.isna().any(axis=0)]
```

```
Out[19]:
```

	value1	value2
0	1.0	4.0
1	2.0	5.0
2	3.0	NaN
3	NaN	6.0

```
In [20]: # Files amb nuls  
df.isna().any(axis=1)
```

```
Out[20]: 0    False  
1    False  
2     True  
3     True  
dtype: bool
```

```
In [21]: # Comprovar files amb nuls  
df[df.isna().any(axis=1)]
```

```
Out[21]:
```

	key	value1	value2
2	C	3.0	NaN
3	D	NaN	6.0

Eliminar nuls

```
In [22]: df_with_no_nans = df.dropna()  
df_with_no_nans
```

```
Out[22]:
```

	key	value1	value2
0	A	1.0	4.0
1	B	2.0	5.0

Omplir nuls

```
In [23]: # Omplir els valors absents amb un 0  
df.fillna(0)
```

```
Out[23]:
```

	key	value1	value2
0	A	1.0	4.0
1	B	2.0	5.0
2	C	3.0	0.0
3	D	0.0	6.0

Datetime

```
In [24]: store = {  
    "Date": ["2019-01-01 20:00", "2019-01-02 20:00", "2019-02-03 20:00", "2019-02-04 20:00"],  
    "Sales": [1000, 1200, 1300, 1100],  
}
```

```
In [25]: # Converteix el diccionari a dataframe  
df = pd.DataFrame(store)  
df
```

```
Out[25]:
```

	Date	Sales
0	2019-01-01 20:00	1000
1	2019-01-02 20:00	1200
2	2019-02-03 20:00	1300
3	2019-02-04 20:00	1100

```
In [26]: print(df['Date'].dtype)

# Converteix la columna de string Date a datetime
df['Date'] = pd.to_datetime(df['Date'])

print(df['Date'].dtype)
```

```
object
datetime64[ns]
```

Aplica funcions de datetime, per exemple:

```
In [27]: # Noms dels dies de la setmana  
df['Date'].dt.day_name()
```

```
Out[27]: 0    Tuesday  
1    Wednesday  
2    Sunday  
3    Monday  
Name: Date, dtype: object
```

```
In [28]: # Obtenir anys  
df['Date'].dt.year
```

```
Out[28]: 0    2019  
1    2019  
2    2019  
3    2019  
Name: Date, dtype: int32
```

```
In [29]: # 0 mesos  
df['Date'].dt.month
```

```
Out[29]:
```

0	1
1	1
2	2
3	2

Name: Date, dtype: int32

```
In [30]: # 0 L'hora  
df['Date'].dt.hour
```

```
Out[30]:
```

0	20
1	20
2	20
3	20

Name: Date, dtype: int32

```
In [31]: # Afegir temps a cada instància  
df['Date'] + pd.Timedelta(hours=1)
```

```
Out[31]: 0    2019-01-01 21:00:00  
1    2019-01-02 21:00:00  
2    2019-02-03 21:00:00  
3    2019-02-04 21:00:00  
Name: Date, dtype: datetime64[ns]
```

Gràfics

Podem mostrar gràfics amb `pandas`, que utilitza `matplotlib` per sota.

```
In [33]: # Gràfic simple de sèrie temporal
df['Sales'].plot(figsize=(5,3), marker='o',
                 linestyle='--', color='red')
plt.title("Vendes al llarg del temps")
plt.xlabel("Data")
plt.ylabel("Quantitat de vendes")
plt.show()
```

